

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4
«Технический дизайн микросервисной архитектуры»

Выполнила:

Гайдук Алина

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service.
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом.
3. Спроектируйте разделение вашей БД на обособленные БД.
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI.
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки.
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы.

Ход работы

Я сформулировала границы пяти микросервисов и API Gateway по следующим принципам:

1. Один микросервис – одна база данных. Между разными базами нет внешних ключей, только логические идентификаторы и проверки через HTTP API.
2. Справочники вынесены в отдельный сервис: стабильные id, свой цикл наполнения, ссылки из рецептов только по id.
3. Подписки пользователь–пользователь (follows) отделены от лайков и комментариев к контенту: разный смысл связей и разная нагрузка на чтение.
4. Таблица follows переносится вместе с пользователями в identity-service, чтобы проверки пар (follower_id, following_id) оставались в одном контексте с users.

В итоге у меня получилась следующая декомпозиция сервисов:

1. identity-service: регистрация, вход, refresh JWT; пользователи; публичный профиль; социальный граф подписок (follows).

2. catalog-service: справочники для рецептов: типы блюд, сложность, теги, ингредиенты, единицы измерения.
3. recipe-service: рецепт: заголовок, шаги, состав со ссылками на id каталога, обложка, время, теги.
4. blog-service: посты блога.
5. engagement-service: комментарии к рецепту и посту, лайки рецепта и поста, сохранённые рецепты.

Далее я составила список маршрутов, которые относятся к каждому владельцу:

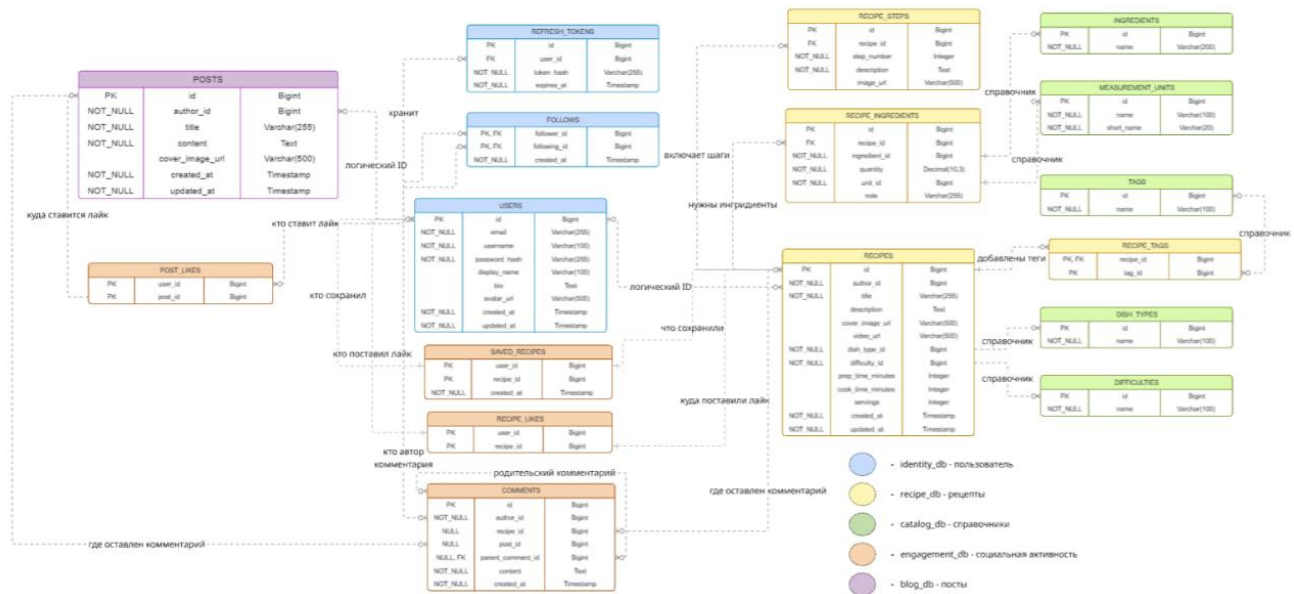
1. identity-service: POST /auth/register, POST /auth/login, POST /auth/refresh, PATCH /users/me, GET /users/{id}, GET /users/{id}/followers, GET /users/{id}/following, POST и DELETE /users/{id}/follow
2. recipe-service: GET /users/{id}/recipes (с проверкой существования пользователя через внутренний вызов к identity), GET, POST, PATCH, DELETE /recipes, GET /recipes/{id}
3. blog-service: GET /users/{id}/posts, GET, POST, PATCH, DELETE /posts, GET /posts/{id}
4. engagement-service: комментарии под рецептом и постом, DELETE /comments/{id}, лайки рецепта и поста, сохранение рецепта, GET /users/me/saved
5. catalog-service: /dish-types, /difficulties, /tags, /ingredients

Далее я перешла к полноценному разделению баз данных, в результате у меня получилось пять независимых БД:

1. identity_db, содержит таблицы users (учётные и профильные данные), refresh_tokens (рефреш токены), follows (ссылки на пользователей, подписчики и подписки);
2. catalog_db, содержит таблицы dish_types (типы блюд), difficulties (уровни сложности), tags (теги), ingredients (ингредиенты), measurement_units (единицы измерения);
3. recipe_db, содержит таблицы recipes (сами рецепты), recipe_steps (шаги рецепта), recipe_ingredients (ингредиенты рецепта), связь recipe_tags (рецепт - тег)
4. blog_db, содержит таблицу posts с логическим author_id (ссылка на users)

- engagement_db, содержит таблицы comments (к рецепту или к посту по id, parent_comment_id внутри той же базы), recipe_likes, post_likes, saved_recipes.

ERD-схему новой модели данных можно посмотреть ниже:



Основными механизмами взаимодействия микросервисов будут синхронный REST, JSON, путь /internal/v1, с помощью них будут осуществляться проверки существования сущностей, валидация справочников, пакетные профили и счётчики.

Для того, чтобы реализовать разделение текущего монолита на микросервисы, необходимо сделать следующее:

- выделить catalog-service и базу catalog_db;
- выделить identity-service с users, refresh_tokens, follows и маршрутами auth и пользователь/social;
- выделить recipe-service и blog-service с переносом таблиц и HTTP-обработчиков;
- выделить engagement-service; подключить внутренние клиенты к identity, recipe, blog;
- ввести API Gateway с маршрутизацией /api/v1 и сохранением совместимости с публичным OpenAPI монолита;
- выполнить миграцию данных и прогнать регрессионные сценарии.

Вывод

Я спроектировала архитектуру из пяти микросервисов и API Gateway с разделением данных по принципу database-per-service. Границы сервисов разделяют учётные записи и подписки (identity), справочники (catalog), контент рецептов и постов (recipe и blog) и активность вокруг контента (engagement). Для меня оказалось важным, что после распила нельзя одним SQL собрать профиль или список рецептов с метриками, вместо этого нужна сеть согласованных синхронных вызовов и явные контракты.